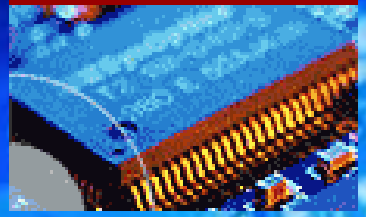
EEE3096S



Embedded Systems II

Development & Execution Environment: Cross-Compiling Toolchain, Make, StdC, Object Files

Lecture L3

Embedded Systems II

Dr Simon Winberg

Electrical Engineering
University of Cape Town

Outline

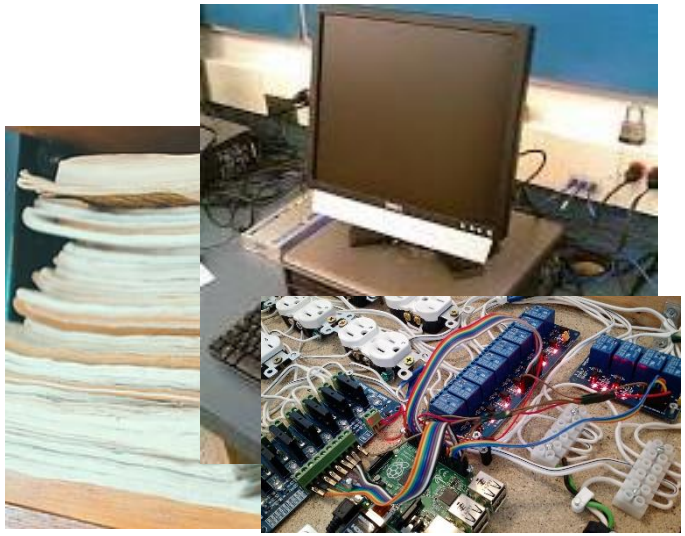
- The Development Environment
- The Execution Environment
- C Run Time Libraries & StdC
- Bintools, Make Files, Executable formats

The Development Environment

The **development environment** =
The environment where development occurs

More specifically it comprises:

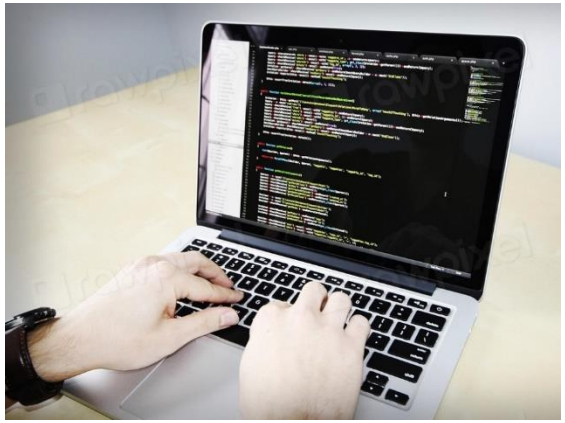
- Equipment used for development
- Tools used for development
  - e.g.: Compilers, Linkers, etc.
- Other Resources
  - e.g.: Manuals, Datasheets, etc.



The PC Dev. Environment

The PC application development environment is:

- Well defined,
- Uses standard tools



These tools depend on an OS, **but not on a particular machine**.

There are **many benefits** to this:

- Toolchain is fully configured and optimised
- Debugging is made easy with an IDE
- Full set of support libraries available
- Simply use the tools and things work (usually)

image source: https://www.rawpixel.com (CC0 public domain)

The Embedded Dev. Environment

- The development environment for ES is generally not so well defined…
- Tools depend on the software vendor, the hardware vendor, peripherals used, etc., & limited combinations of these*
- Toolchain often built from scratch
- Debugging needs special techniques
- Few, if any, support libraries may be available
- Significant time often spent in preparing a custom development environment

A tidier workspace is probably an easier one to work in…

A bit more mentioned on each of these…

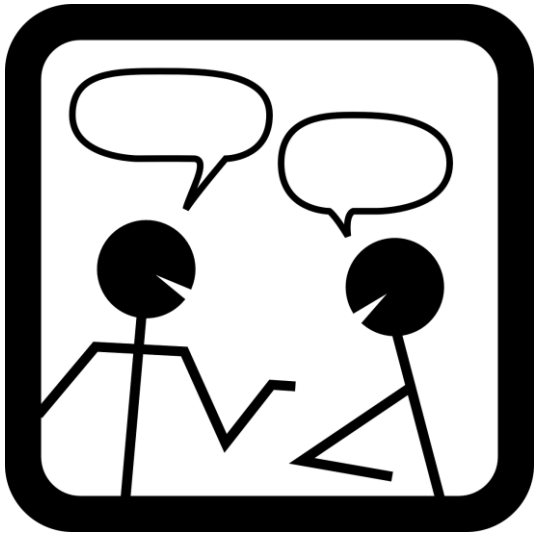
* Why do I add this disclaimer? Because sometimes your selected tool supports only certain peripherals for a particular platform, or certain processors for a given compiler etc.

The Embedded Dev. Environment

Toolchains are often (partly) built from scratch

- Sure, many processor vendors provide a comprehensive set of tools… e.g. compilers, debuggers, etc…
- But **often additional tools are needed**. e.g. tools to
  - Configure peripherals (e.g. to configure radio frequency front-end)
  - Programme memory devices (e.g. EEPROM programmer)
  - Generate data system needed (e.g generate tx signals for a radar)
  - Some of these tools you may need to built yourself, some may be purchased or available as open-source solution

Moment of Chat



thinking point …

Question (understanding check)

Q: By stating “I developed my own adapted toolchain” does this mean …

1. You built all the tools in the toolchain from scratch?
2. You build some of the tools yourself?
3. You’re probably using mostly existing tools but have made some modifications to the tools and how they are used.

Might your use of STM32CUBEIDE relate slightly to this idea?

ANSWER & Elaboration on Tool Chain

- ANSWER: 3 is the best answer.

Generally, developers don't build an entire toolchain from scratch (of course sometimes **big** ones do, like Microchip, Intel, ARM, etc.).

Rather (put more clearly) to use:

Adapted Toolchain =
Using mostly pre-existing tools used, but doing some adjustments to the standard toolchain, e.g. changing configuration settings, setting up memory map for specialized platform used, etc. Maybe some small changes to the actual tools (like adjusting parts of the compiler, if have the sources available)

Custom Toolchain =
Many of the tools and configurations have been done from scratch (or at least significant recoding of multiple tools and scripts used in original toolchain).

ANSWER & Elaboration (cont.)

And what of …

Might your use of STM32CUBEIDE relate slightly to this idea?

Yes, in some sense, but it is not so much modification of tools, rather *configuration* of them, which can involve many settings and get complicated … in that sense we use an adapted toolchain since you can't really do much with a standard STM32CUBE installation without configuring it for a platform.

The Embedded Dev. Environment

- Thusly, **some tools** in a toolchain used by an ES dev team are often built from scratch
- Furthermore, the **platform** being used may be rather **unique** (e.g. custom FPGA or SoC processor solution)…
  - Debugging needing special techniques
  - Few, if any, support libraries available
  - Significant time spent preparing a custom development environment

Your experienced co-worker might say:
"Simply use the tools"… and you discover there's nothing particularly *simple* about it

Relevance to the Engineer

- **Compared to a PC developer**, the embedded systems developer must:
  - **Organize & maintain** the development environment for the target platform
  - **Know more about** the development tools, which they need to configure and build
  - Know more about **underlying hardware** to configure the tools and write drivers
  - All above is important during design and earlier steps (e.g., deciding partitioning decision)

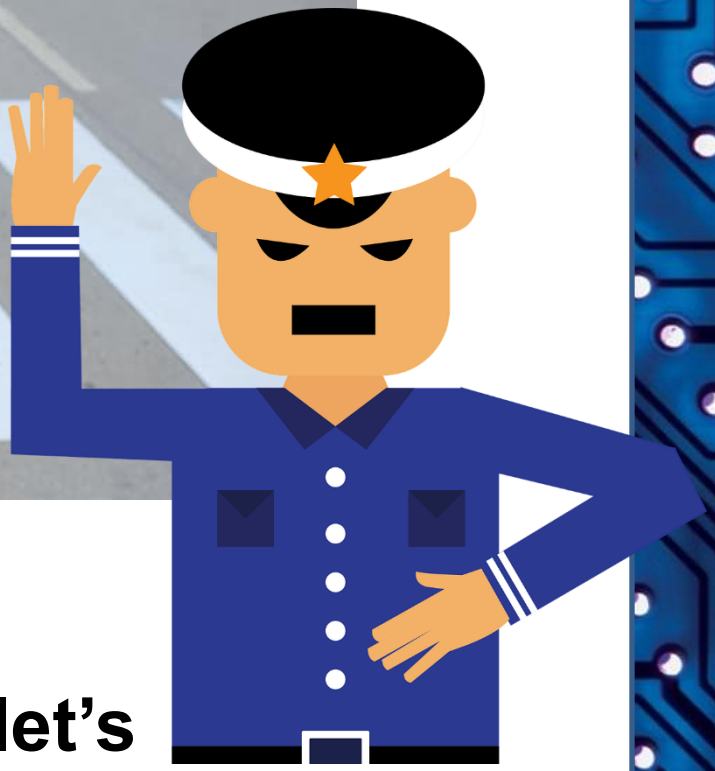
Deciding the

Execution Environment & Compiling Toolchain

for the

Development Environment

I know, we're getting washed away with theory, we need an anchor …



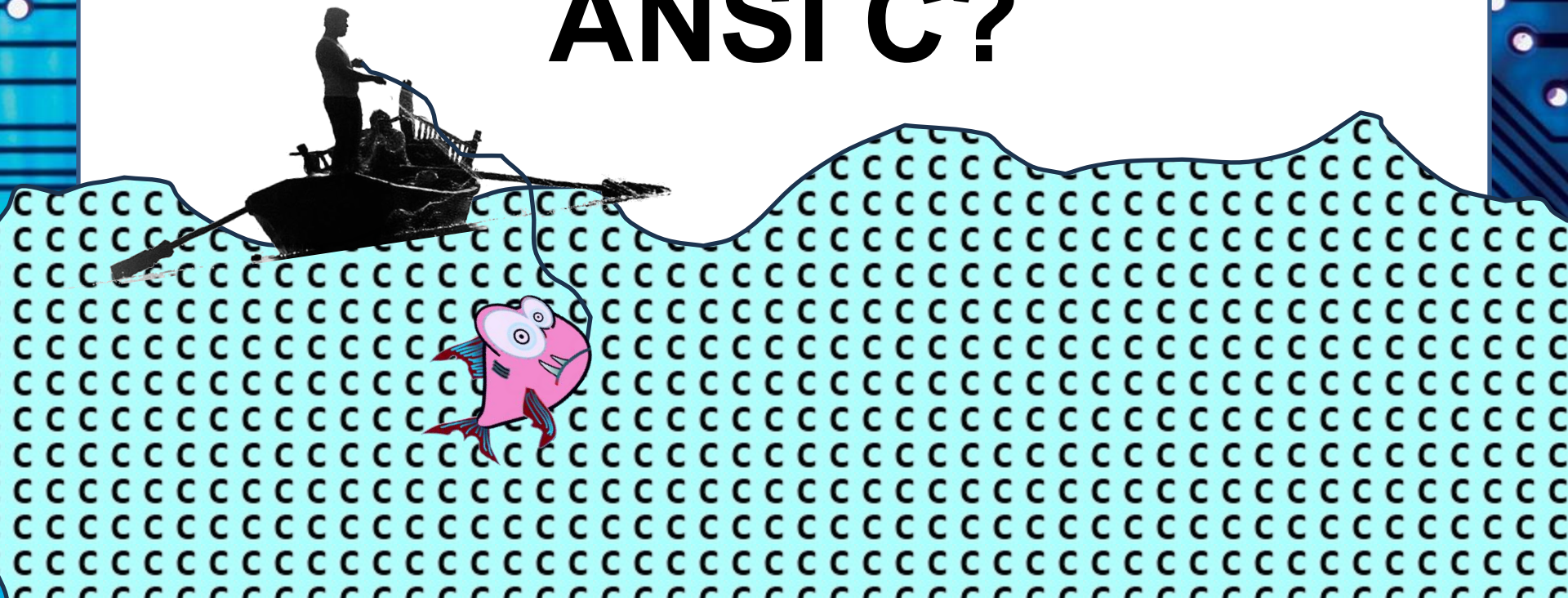
But wait!

Before we get too theoretical, let's rather choose a language of choice to make discussion more practical…

The programming language to choose…

Why is it so often

ANSI C?



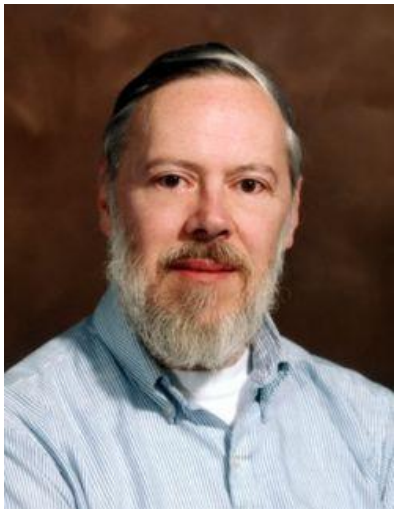
What is C?

I know that you already know what it is and have used it …

… but have you thought why it is still so commonly used?

- **Popular, powerful and flexible** programming language
- The “lowest common denominator” – C is extremely simple, without sacrificing flexibility.
- Some people refer to C as a “portable assembly language”. It is extremely close to the hardware.
- C offers a very low level of abstraction compared to Java, Python, etc.
- C++ is based on C and C++ had C compatibility as one of its main design goals.

The History of C



Dennis Ritchie

The C Father

- C was **developed at AT&T Bell labs** in 1960s - 1970s, culminating in first main version in 1972.
  Led primarily by Dennis Ritchie.
- C was developed as a language to use to write Unix (a point many students don't known nowadays).
- C was intended to be a language for the development of *system software** (rather than *application software – which was novel in its time*).
- For C to be useful for writing operating systems, C needed to be *high level* enough to be portable, while *low level* enough to get sufficient control over hardware…

This is what makes it so useful for embedded software!

**See definition for system software vs. application software*

Why ANSI (ISO) C?

- C offers the embedded systems programmer ***just enough* abstraction**.
- Enough abstraction to write **portable code**.
- Enough abstraction to make writing (and reading) code simple.
- *Not too much* abstraction, to allow efficient access to the hardware without burdensome wrappings.
- *Especially* as C is extremely popular and widely supported on nearly every micro-processor/controller.
- The C language is powerful, flexible and expressive.

main reasons for ES

Consider a look at Rust:
https://www.rust-lang.org/
Has some similarity to C, benefits allowing more rapid, reliable code development

ANSI = American National Standards Institute
ISO = International Organization for Standardization

skip slide
(added background)

Why Not [much] C++?

- C++ has not seen as widespread use in embedded system development as has standard C.
- C++ contains a lot of baggage and language constructs which can cause bloat.
- C++ programs are (depending on style) less efficient than C programs.
- Most simply: C++ just doesn't have the same very wide compiler support for embedded system platforms.

This situation is changing. Compilers are improving, memories are getting larger and C++ is gaining "mindshare" *.

* An important issue since engineering (NRE) cost is often a big part of the product cost

skip slide
(suggested links)

C Tutorials (useful recapping)

- You already know C from ES1 and possibly C++ in comp. science courses.
- *However*, if you're one of the few students that haven't already done C, or you need a refresher, then here's two recommended online tutorials, which you can choose depending on whether you want the convenience of doing it online or if you prefer a stand-along option:

ONLINE OPTION

If you just want a quick refresher and don't want to spend time setting up tools or copying the code examples, then this is an excellent option:

https://www.learn-c.org/

OFFLINE OPTION

If you want to download things and do the exercises offline, then follow these tutorials (note that these are more thorough and go a bit deeper than the above online option):

https://www.cprogramming.com/tutorial/c/lesson1.html

… back to Execution Environment & Compiling Toolchain issues

The Execution Environment

Also commonly called [and sometimes incorrectly so, for some systems] the "runtime environment" (RTEnv) or "runtime system" (RTS) *

[note use of RTE and RTS can obviously cause confusion with realtime embedded and realtime system, I recommend using 'RTEnv' (not RTE) for RunTime Environment to be clear]

Let's clarify the important distinction RTEnv vs. RTS …

RTEnv vs RTS : clarifying the differences

- The Runtime Environment (RTEnv) and Runtime System (RTS) are not the same. The RTS is a core component inside the RTEnv.
- Runtime System (RTS) — The Engine 
  - A set of routines that implements core language features like garbage collection, memory management, and thread scheduling.
- Runtime Environment (RTEnv) — The Entire Car 
  - The complete platform or sandbox where your program executes.
  - Provides all necessary components for code to run, including RTS, standard libraries, and access to system resources (like peripherals or IPC).
  - Examples: Node.js, Java Virtual Machine (JVM), .NET

The Execution Environment

- To write software for embedded systems, you must know the details of:
  - System memory layout, including the stack and free memory
  - What happens at system start-up
  - How the system handles interrupts
  - How the system handles exceptions

Which is essentially knowing your 'execution environment'

which we will define more formally as…

Execution Environment = RTEnv where embedded program (not dev. tools) runs

Execution Environment (‘EE’)

- The term ‘execution environment’ (EE) refers to those components that are used together with the application’s code to make a complete computing system, including: the processors, networks, operating systems and so on.
- A programming language has some type of computer-based environment, called the ‘runtime environment’ (RTEnv) or ‘runtime system’ for which the resultant program is modelled to run in and to connect with.
- The runtime environment should, more accurately, be considered a large part of the ‘execution environment’…
- In reality, especially for large systems where there are multiple languages and processes used (e.g. Python, C and CUDA) the execution environment may comprise multiple runtime environments – hence EE is a superset of RTEnv.

Runtime Environment (RTEnv)

supplementary

- This 'runtime environment' (RTEnv) addresses various issues, especially the aspects of:
  - How the program starts
  - How procedures/functions are called and results returned
  - Methods for passing parameters between procedures
  - How the program accesses or manipulates variables
  - Management of application memory
  - Ways to interface to the operating system
  - Inter-process communication (IPC) of the platform [if OS supports this]
  - How to connect to peripherals or to do I/O (note that an OS might not actually support direct control of I/O port control, but only provide higher-level functions like write a data type A to device X)
- The compiler makes assumptions depending on the specific runtime system selected for which executables are to be generated.
- The RTEnv *usually* (not always!) has responsibility for setting up and managing the stack and heap, and may include features such as garbage collection, threads or other dynamic features provided by the language. (if you're using a very basic compiler or assembler, essentially just converting asm code to instructions and allocating blocks of memory, you would need to explicitly implement stacks or heaps if your application needs them – you may do this regardless for processors with very limited memory)

PC vs Embedded RTEnv

- The **RTEnv is well standardised for the PC**
  - POSIX, Win32 and other API standards
  - Operating Systems have ABI* standards
- It's not so well defined on embedded systems
  - Often no OS[+] to provide neat API standard
  - ABI depends on exact system configuration

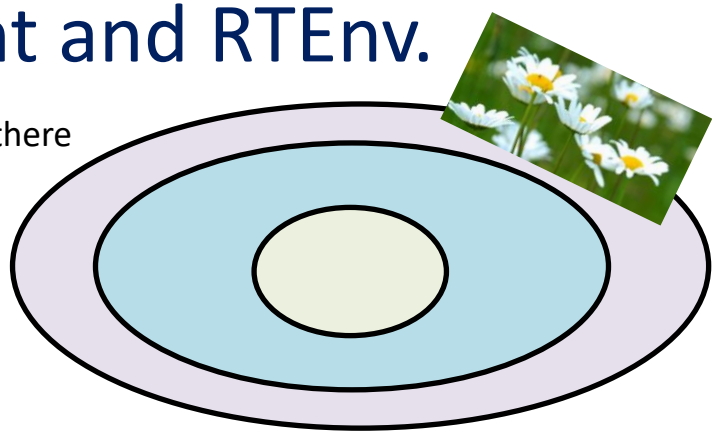
*** application binary interface** (**ABI**) defines the low-level interface between programs or between a program and the operating system. An ABI standard defines sizes and structures of data types, endianness, system calls numbers or Operating System (OS) connection, calling convention, format of executable files, etc.
Example: Intel Binary Compatibility Standard (iBCS).
For more detail see: http://en.wikipedia.org/wiki/Application_binary_interface

[+] the term 'running on bare metal' traditionally means running without an OS.

The Layers of Environments

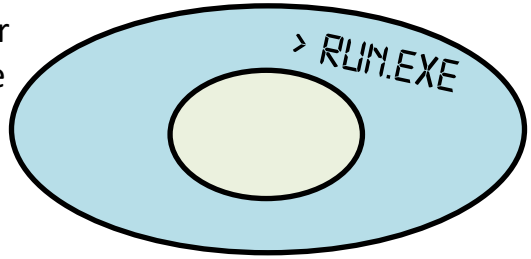
- This diagram is aimed to help clarify the different between Runtime System, Execution Environment and RTEnv.

During development and use, there may be multiple forms of the these environments. Similarly, the developers might not all use the same development environment.

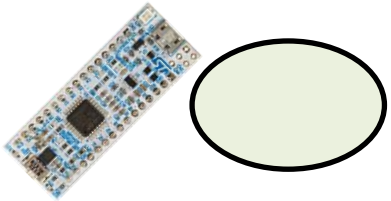


Runtime Environment
An environment in which your application operates and does useful or desired tasks within the environment or a context of where the product is used.

Execution Environment defines **rules, constraints, and services** under which your application & RTS operate on the target. Can be an abstract concept that describes the interface between your software and the underlying system.



Execution Environment
An environment in which your application operates, can run and do things but not necessarily a target environment or context of use.



Runtime System (RTS)
Platform with minimal set of supporting system code to get your application running (not necessarily connecting to environment).

General C Runtime Environment

The
GNU GCC and Binutils
Tools

(getting code executed on a platform)

The Compiling Toolchain

- A software compiling toolchain includes programs to:
  - convert source code into binary machine code
  - link these together
  - Separately assembled/compile code modules
  - Disassemble binaries
  - Convert formats
- The above are the fairly essential (minimal set) of tools needed. Usually additional tools are provided, such as: debugging support (possibly using ICE*), tools to program the platform, profiling tools, etc.

* ICE = In-circuit Emulator, allows you to replace the processor with a equivalent device that provides additional debugging support, e.g. pause processor, download registers, etc.

A Cross-Compiling Toolchain

Defn cross-compiler =
A compiler capable of creating executable code for a platform other than the one on which the compiler is running.

Common Examples:
ARM and PIC compilers that are run on a PC (x86 processor) for generating executables for embedded platforms.

Cross-compiler toolchain is thus the broader collection of tools used in developing software solutions for a platform.

Typically, for your development environment, you start with a baseline toolchain, e.g. arm-gcc, and add additional programs and other tools you need to develop your system.

GCC: GNU Compiler Collection

- This is the GNU project's compiler tool chain. They provide the full source for GCC and documentation explaining how to port it to other processor architectures. So, you could say, GNU's GCC project is a collection of cross-compilers that have a similar runtime environment for which the tools all follow a consistent user interface.

GCC Tools

- cpp: c pre-processor for macros
- cc1: ‘c compiler phase 1’ perform semantic routines and translate C into assembly language
- as: assembler to relocatable object files
- ld: linker
- To view the commands executed to run the stages of compilation (and other verbose output) try running:

```
gcc -v
```

GNU Binutils

supplementary

- The GNU Binutils are a collection of binary tools commonly used with GCC.
- They are used for creating & managing:
  - binary executable programs
  - object files
  - Libraries
  - profile data, and
  - assembly code
- The most commonly used ones are:
  - ld : the GNU linker
  - as : the GNU assembler

GNU Binutils – List of Main Tools

supplementary

Source: https://en.wikipedia.org/wiki/GNU_Binutils

Command	Description
as	assembler popularly known as GAS (GNU Assembler)
ld	Linker (*collect2* is a higher-level application that uses ld)
gprof	profiler
addr2line	convert address to file and line
ar	create, modify, and extract from archives
c++filt	demangling filter for C++ symbols
dlltool	creation of Windows dynamic-link libraries
gold	alternative linker for ELF files
nlmconv	object file conversion to a NetWare Loadable Module
nm	list symbols in object files
objcopy	copy object files, possibly making changes
objdump	dump information about object files
ranlib	generate indices for archives (for compatibility; same as ar -s)
readelf	display content of ELF files
size	list total and section sizes
strings	list printable strings
strip	remove symbols from an object file
windmc	generates Windows message resources
windres	compiler for Windows resource files

GCC Verbatim Output

supplementary

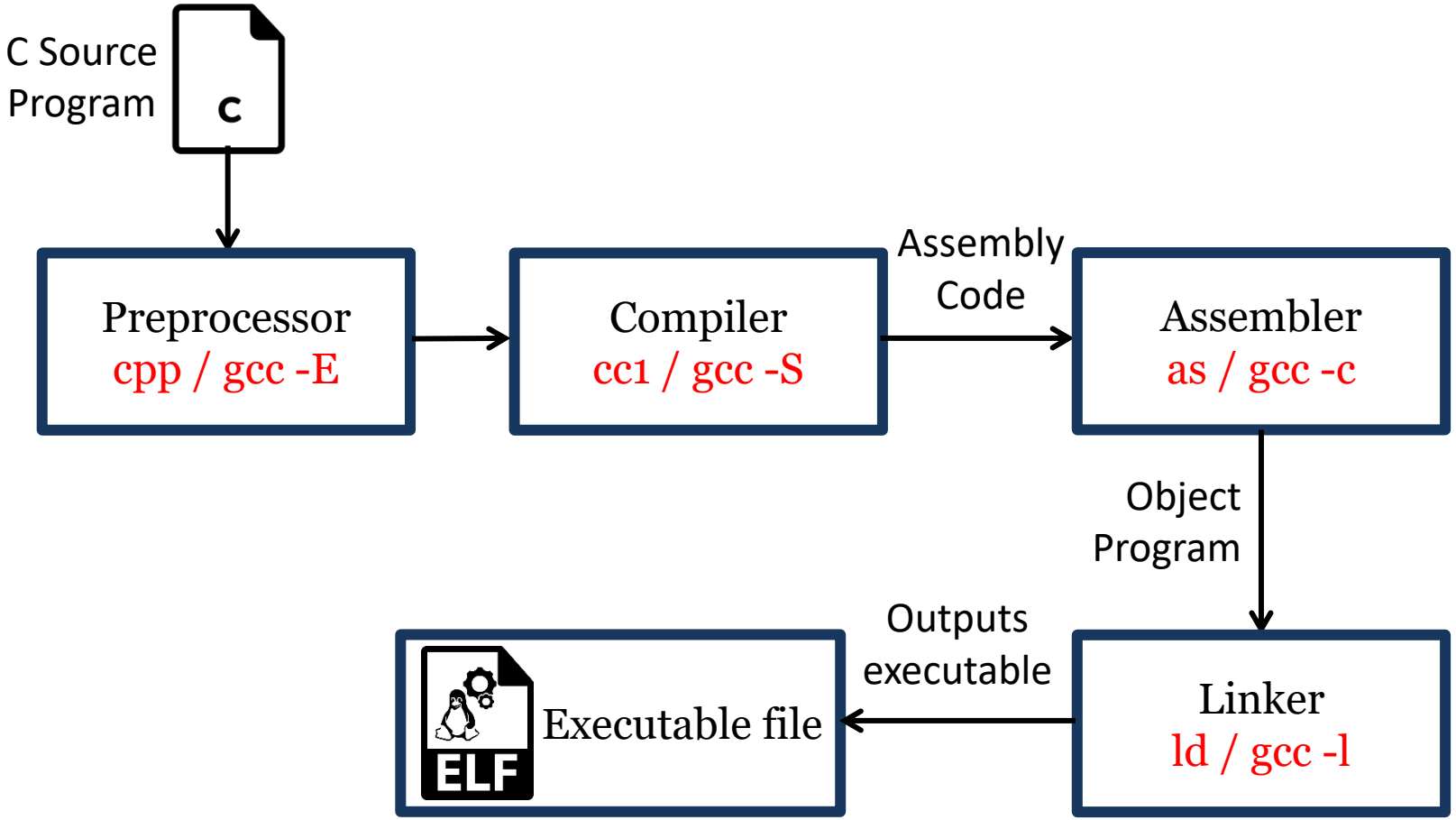
Example of 'gcc -v' console output. Running **gcc –v main.c**


```
Using built-in specs.
COLLECT_GCC=gcc.real
COLLECT_LTO_WRAPPER=/usr/lib/gcc/x86_64-linux-gnu/4.8/lto-wrapper
Target: x86_64-linux-gnu
Configured with: ../src/configure -v --with-pkgversion='Ubuntu 4.8.4-2ubuntu1~14.04.4' --with-bugurl   … etc
…
#include <...> search starts here:
 /usr/lib/gcc/x86_64-linux-gnu/4.8/include
 /usr/local/include
…
cc1 -quiet -v -imultiarch x86_64-linux-gnu main.c -quiet -dumpbase main.c -mtune=generic -march=x86-64 -auxbase main -version -fstack-protector -Wformat -o /tmp/ccGnSEt7.s
…
as -v --64 -o /tmp/ccbDwajP.o /tmp/ccKnqIvv.s
GNU assembler version 2.24 (x86_64-linux-gnu) using BFD version
…
collect2 -m elf_x86_64 /usr/lib/gcc/x86_64-linux-gnu/4.8/../../../x86_64-linux-gnu/crt1.o …
/tmp/ ccGnSEt7.o -lgcc --as-needed  ….
```

This generates the output executable named a.out by default.

Note on collect2: you could effectively replace collect2 with ld. However collect2 is used in many gcc compilers nowadays as it is a more optimized linker, it will tweak the placement of modules, variables etc. to generate more efficient executables.

From C source to executable

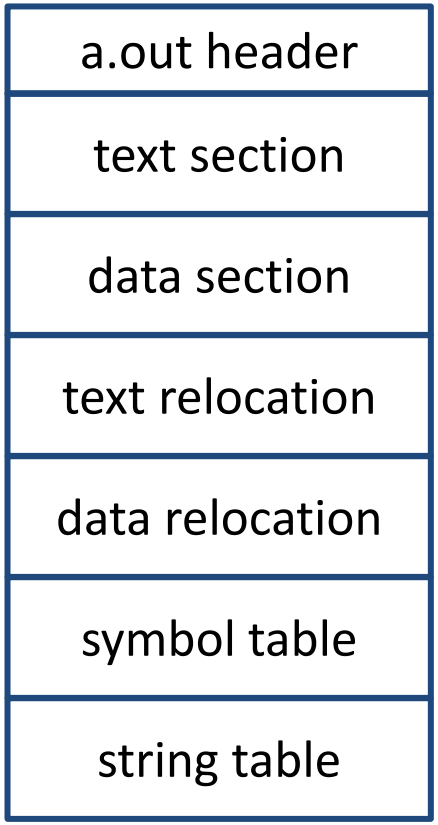


a.out Executable File Format

supplementary

Unix a.out Format

- Hardware Memory Manager Unit (MMU) is needed.
- No relocation at load time (i.e. is a simple absolute format).
- Separate address space for code or instruction (I-space) and data (D-space), each only 64KB.



Elf Executable File Format

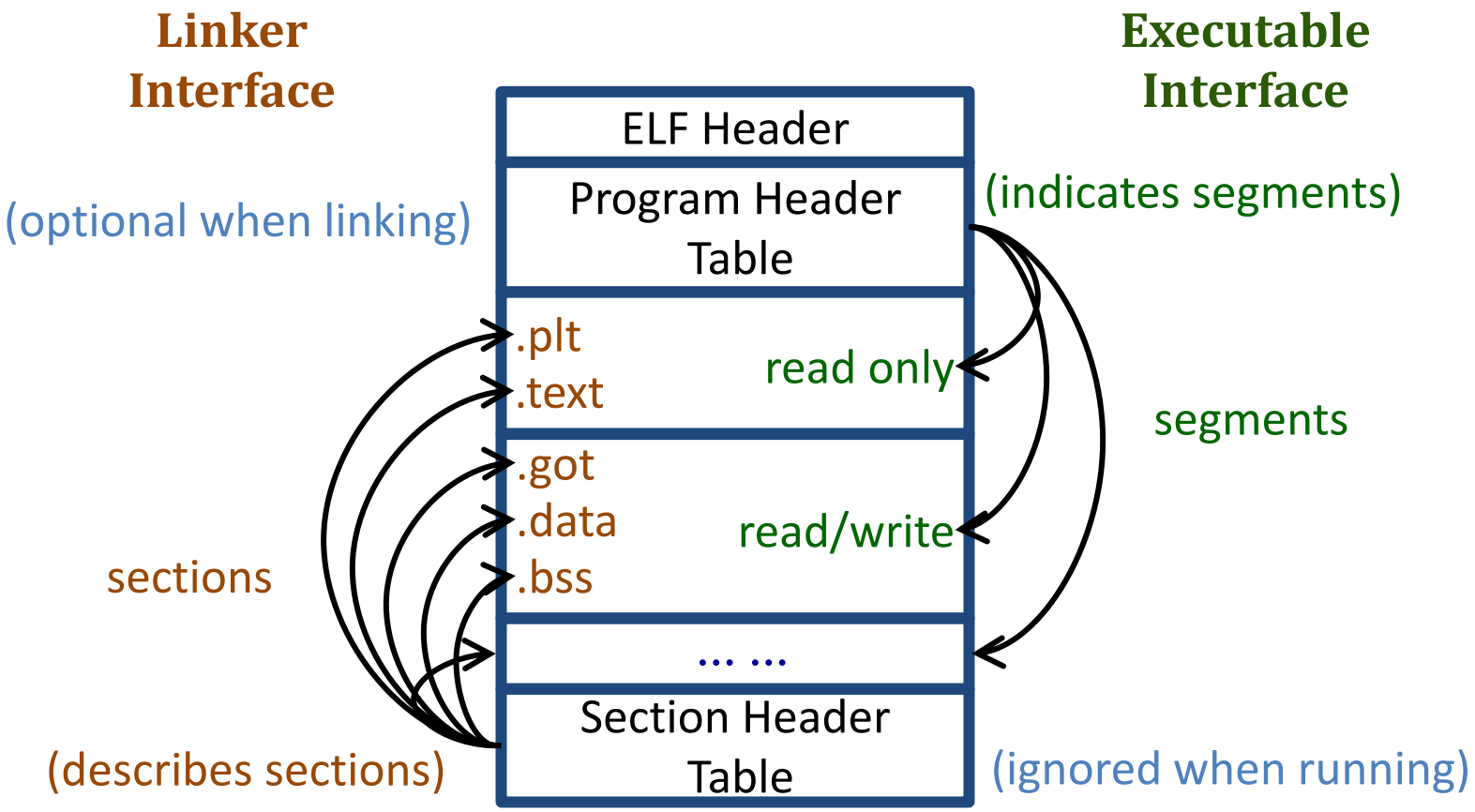
Unix Elf Format

- Designed to support cross-compilation, dynamic linking and other modern system features.
- Three slightly different versions of object file
  - relocatable, (usable as precompiled library)
  - executable, and (can run directly on platform)
  - shared object. (e.g. compiled library, or shared between programs)
- Provides both:
  - A set of logical sections described by section tables. (for compilers, assemblers, and linkers)
  - A set of segments described by a program header table. (for executable loaders)

supplementary

Elf File Structure

supplementary



End of Lecture

Extra Slides / Additional Useful Points

the slides that follow are not in the exam syllabus

Applicability of Java and JVM for the Embedded Dev. Environment
… might it be a solution to the issue of needing different compilers, multiple processors, etc??

these slides not examined

Is the JVM The Solution?

Question:

- Can we solve this problem by moving to a machine independent machine language, such as Java bytecode executed by a Java Virtual Machine (VM)?

Answer:

- It depends largely on the application concerned.
- There are various limitations to existing Java VMs and the Java bytecode structure.
- And (duh... rather obviously) an Embedded System is usually more than just processors; you need to control peripherals and other parts of the system it connects to, even if we all did just use a standard VM, that would solve only part of the problem!

Processor Standardization Issues

- Java problems … *
  - Performance unpredictability
  - Difficulty in writing low level code
  - Difficulty in interfacing with peripherals
  - Difficulty in doing “the last little bit” of optimization (esp. if you don’t know the specific VM implementation in use)
- Standardized processor architectures and machine languages generally preferred (e.g. ARM cores and ARM / ARM Thumb instruction sets)

* Not to say I’m not a fan of Java!! It’s just that one needs to choose the right solution for a given problem, and currently Jave and the JVM is rather often an unnecessarily hard or ineffective solution to a design problem.